

Making Http Calls

Hubot can make HTTP calls on your behalf to integrate & consume third party APIs. This can be through an instance of node-scoped-http-client available at robot.http. The simplest case looks like:

```
jarvis.http("https://midnight-train").get()
(function(err, res, body) {});
```

A post call looks like:

```
var data;
data = JSON.stringify({
  foo: 'bar'});
jarvis.http("https://midnight-train").header('Content-Type',
'application/json').post(data)(function(err, res, body) {
/* Your bot code here */});
```

err is an error encountered on the way, if one was encountered. You'll generally want to check for this and handle accordingly:

```
jarvis.http("https://midnight-train").
get()(function(err, res, body) {
  if (err) {
    res.send("Encountered an error :( " + err);
  });
});
```

res is an instance of node's http.ServerResponse. Most of the methods don't matter as much when using node-scoped-http-client, but of interest are statusCode and getHeader. Use statusCode to check for the HTTP status code, where usually non-200 means something bad happened. Use getHeader for peeking at the header, for example to check for rate limiting:

```
jarvis.http("https://midnight-train").
get()(function(err, res, body) {
  var rateLimitRemaining;
  if (res.statusCode !== 200) {
    res.send("Request
didn't come back HTTP 200 :(");
    return;}
  if (res.getHeader('X-RateLimit-Limit')) {
    rateLimitRemaining =
parseInt(res.getHeader('X-RateLimit-Limit'));
```

```
    if (rateLimitRemaining && rateLimitRemaining < 1) {
      return res.send("Rate
Limit hit, stop believing for awhile");}});
```

body is the response's body as a string, the thing you probably care about the most:

```
jarvis.http("https://midnight-train").
get()(function(err, res, body) {
  return res.send("Got back " + body);});
```

The Hubot Library also supports various other things like custom listeners, creating a topic and a room where you can chat with multiple users. You can have http listeners, persist data and do a lot more stuff.

Combining all of the things we have learned till now, let us create a weather bot that will tell us the average temperature of a place that we are asking for. The key phrase for our bot will be "weather of {{place_name}}". You can further extend this code by adding nlp based parsing support and create intents based on that. Here I'm using openweathermap's api for my purpose.

```
module.exports = function (jarvis) {
  jarvis.hear(/weather of (\w+)/i, function (jar_resp) {
    var city_name = jar_resp.match[1];
    var url = "http://api.openweathermap.org/
data/2.5/weather?q=" + city_name +
"&appid=" + api_key + "&units=metric";
    jarvis.http(url).get()(function (err, res, body) {
      jar_resp.send("The average temperature of " + city_name
+ " at this moment is " + JSON.parse(body).main.temp + "°C");
    });
  });
};
```

Conclusion

The bot space is currently growing in the world and many other players are there to help you with your bot making. This is one way to make bots but other companies like Gupshup, Glomantra, Congnicor, Magicx, Arya.ai etc are helping users to make bot without having the knowledge of any language and deploy across multiple channel with the help of a button click. And pretty soon, almost every conceivable action that needs you to interact with a human being to obtain certain services - it will all be done by bots before you know it. >

*Nodes of Interest



*Official Github page for Hubot

>> Here you will find the entire source code as well as documentation for Hubot.

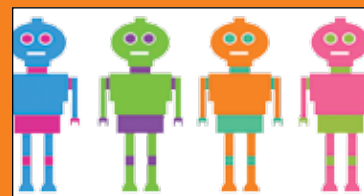
<http://dgit.in/HubotGithub>



*Chatbots magazine

>> A website that covers everything related to chatbots and platforms that use them like Slack, Facebook etc and the tech behind chatbots.

<http://dgit.in/ChtBtMgzn>



*Getting started with Hubot

>> Another Hubot tutorial that is basic and easy to follow to get you started on it.

<http://dgit.in/HubotStartrs>